



BITS Pilani

Pilani | Dubai | Goa | Hyderabad

Artificial and Computational Intelligence

AIMLCZG557

ACI Team



BITS Pilani

Pilani | Dubai | Goa | Hyderabad

AIMLCZG557 – CS#14

Multi Agent Decision Making

Introduction



- Artificial Intelligence is no longer limited to solving problems in isolation.
- In many real-world applications, multiple intelligent agents interact, cooperate, or compete to achieve their objectives. Such scenarios require decision-making based not only on the environment but also on predicting the actions of other intelligent agents.
- Game Theory provides the mathematical foundation for analyzing these strategic interactions.
- It enables AI systems to make optimal decisions in environments where the outcome depends on the choices of multiple participants.

Applications

- Chess and Board Games
- Autonomous Vehicles
- Robot Soccer
- Cybersecurity
- Business Negotiation
- Online Auctions
- Financial Trading
- Multi-Robot Systems

How can an AI agent choose the best action when another intelligent agent is trying to influence the outcome?

Learning Objectives



After completing this chapter, students will be able to:

Knowledge

- Define Game Theory and explain its importance in AI.
- Identify the components of a strategic game.
- Differentiate between cooperative and non-cooperative games.
- Explain zero-sum and non-zero-sum games.

Understanding

- Describe rational decision making.
- Explain how intelligent agents reason about opponents.

Learning Objectives



After completing this chapter, students will be able to:

Application

- Construct simple game trees.
- Apply the Minimax principle.
- Analyze repeated and sequential games.

Higher-Order Thinking

- Evaluate optimal strategies.
- Predict opponent behavior.
- Analyze Nash Equilibrium in strategic interactions.

Why Do We Need Game Theory in AI?



Most AI algorithms assume that the environment is passive.

Examples

- Path planning
- Image classification
- Speech recognition

However, **many AI problems involve active opponents or other intelligent agents.**

Examples

- **Chess:** The opponent continuously changes the game state.
- **Autonomous Driving:** Each vehicle makes independent decisions.
- **Online Auctions:** Every bidder changes the market price.

Why Do We Need Game Theory in AI?



Cybersecurity

- Attackers and defenders constantly adapt to each other's strategies.
- In such situations, the success of an AI agent depends not only on its own decisions but also on accurately anticipating the behavior of others.

Therefore,

Game Theory helps AI agents answer three fundamental questions:

- What should I do?
- What will my opponent do?
- What is the best possible response?

What is Game Theory?



- Game Theory is the mathematical study of strategic decision making among multiple rational agents whose outcomes depend on the actions chosen by all participants.
- Unlike optimization problems involving a single decision maker, Game Theory considers situations where every player's decision affects every other player's outcome.

Components of Game Theory

- Multiple Players
- Set of Available Actions
- Strategies
- Payoffs (Utilities)
- Information available to each player

What is Game Theory?



Important Idea

- Every player is assumed to be **rational**.
- A rational player selects actions that maximize their own benefit while considering the possible decisions of other players.

Example

Suppose two companies launch similar products.

Company A must decide its price.

Company B is making the same decision simultaneously.

The profit of each company depends on **both pricing decisions**, not just its own.

This is a strategic game.

Elements of a Game



Every game consists of five important elements.

- Players
- Actions
- Strategy
- Payoff (Utility)
- Information

1. Players

Decision makers participating in the game.

Examples

Human players

AI agents

Companies

Robots

Elements of a Game



2. Actions

Possible moves available to each player.

Examples

Move Left

Raise

Fold

Attack

Defend

3. Strategy

A complete plan describing what action a player will take in every possible situation.

A strategy is more than a single move—it specifies how to act throughout the game.

Elements of a Game



4. Payoff (Utility)

The reward obtained after the game ends.

Examples

Winning

Profit

Score

Utility Value

5. Information

The knowledge available to a player before making a decision.

Players may have:

Complete Information

Partial Information

Imperfect Information

Think About It

Can two players choose the same action but receive different payoffs?

Yes, because the payoff depends on the combination of actions chosen by all players.

Characteristics of Rational Agents



A rational agent always attempts to maximize its expected utility.

A rational agent:

- Selects the best available action.
- Predicts opponent behavior.
- Plans several steps ahead.
- Evaluates future consequences.
- Avoids actions leading to lower utility.

Assumptions Used in Game Theory

Players:

- Know the rules.
- Know their available actions.
- Prefer higher utility.
- Make logical decisions.
- Never intentionally choose a worse outcome if a better one exists.

Important Observation

- Rational does **not** mean selfish.
- A rational player may cooperate if cooperation results in a higher payoff.

Types of Games



Games can be classified in several ways.

- Zero-Sum Games
- Non-Zero-Sum Games
- Cooperative Games
- Non-Cooperative Games

Zero-Sum Games

One player's gain equals another player's loss.

Total payoff remains constant.

Examples

- Chess
- Tic Tac Toe
- Checkers

Winner gains exactly what the loser loses.

Types of Games



Non-Zero-Sum Games

Both players may gain.

Both players may lose.

Examples

- Trade Negotiation
- Climate Agreements
- Traffic Management

Cooperative Games

Players communicate and work together.

Examples

- Multi-Robot Coordination
- Team Sports

Types of Games



Non-Cooperative Games

Players act independently.

Examples

- Chess
- Poker
- Online Auctions

Which category does Poker belong to?

Answer: Non-Cooperative, Imperfect Information

Representing Games



Different game representations are suitable for different types of problems.

1. Payoff Matrix
2. Game Tree
3. Extensive Form
4. Finite State Machines

1. Payoff Matrix

Used when players act simultaneously.

Best suited for simple strategic games.

Example

	B1	B2
A1	3	-1
A2	2	4

Representing Games



2. Game Tree

Represents sequential decision making.

Shows every possible move from beginning to end.

3. Extensive Form

Displays

- Decision nodes
- Chance nodes
- Information sets
- Utilities

Useful for sequential games.

Representing Games



4. Finite State Machines

Represents repeated strategies.

Useful in repeated Prisoner's Dilemma.

Nash Equilibrium - The Prisoner's Dilemma



In this game, we have two prisoners, Alan and Ben, who were caught for the same crime and are held in two different interrogation rooms. They've been given two choices:

- Stay silent
- Confess to the crime

Let's say that each of them is given two choices. So, there would be 4 outcomes in total:

- {Silent, Silent}
- {Confess, Silent}
- {Silent, Confess}
- {Confess, Confess}

And these 4 outcomes can be conveniently represented as a game matrix:

		Ben	
		Silent	Confess
Alan	Silent	A:-1, B:-1	A:-15, B:0
	Confess	A:0, B:-15	A:-10, B:-10



Following the results of the outcome:

- 1.If both of them remain silent, both of them get imprisonment for a year
- 2.If either one of them confesses, the confessor walks free and the other prisoner gets 15 years of imprisonment
- 3.If both of them confess, both of them receive imprisonment for 10 years

The dilemma comes in because neither prisoner is aware of what the other prisoner did. So, what do you think is the Nash Equilibrium action in this problem? *It is very intuitive to think that prisoners would collaborate with each other and stay silent.*

Nash Equilibrium - The Prisoner's Dilemma



But then we also know that prisoners have evident self-interest in minimizing the imprisonment they receive. And even if they remain silent, they still get imprisonment of a year each. So what actually happens is this:

		Ben	
		Silent	Confess
Alan	Silent	A:-1, B:-1	A:-15, B:0
	Confess	A:0, B:-15	A:-10, B:-10



Nash Equilibrium - The Prisoner's Dilemma



Now, this is something Ben would also be thinking. If we focus on the game matrix, the thinking process would make perfect sense:

		Ben <i>confesses</i>				Ben <i>stays silent</i>	
		Silent	Confess			Silent	Confess
Alan	Silent	A:-1, B:-1	A:-15, B:0	Alan	Silent	A:-1, B:-1	A:-15, B:0
	Confess	A:0, B:-15	A:-10, B:-10		Confess	A:0, B:-15	A:-10, B:-10

- In the case where Ben confesses, Alan's best choice is to confess. This will lead to lesser punishment of 10 years rather than 15
- In case Ben stays silent, Alan is still better off confessing as he will be a free man instead of facing one-year imprisonment if he also stays silent.

So this game matrix is in perfect congruence with what Alan is thinking. Now, if Ben is also thinking the same, the Game matrix would look like this for him:

Nash Equilibrium - The Prisoner's Dilemma



		Ben	
		Silent	Confess
Alan stays silent	Silent	A:-1, B:-1 → A:-15, B:0	
	Confess	A:0, B:-15	A:-10, B:-10
		Ben	
		Silent	Confess
Alan confesses	Silent	A:-1, B:-1	A:-15, B:0
	Confess	A:0, B:-15 → A:-10, B:-10	

- Let's say that Ben also goes through the rational thought process as Alan. Ben also comes to the conclusion that no matter what Alan chooses, he will always benefit from confessing. Now, if we superimpose the Rational thinking of both these prisoners, the result is something like this:

		Ben	
		Silent	Confess
Alan	Silent	A:-1, B:-1 → A:-15, B:0	
	Confess	A:0, B:-15 → A:-10, B:-10	
		Alan	

- And looking at the results, the best strategy comes out to be {Confess, Confess}. Even if either of them tries to deviate from this action, they are worse off than what they are getting by playing this action. **Hence, {Confess, Confess} is a Nash Equilibrium strategy.**

Nash Equilibrium - The Prisoner's Dilemma



- Makes perfect sense, right? For Nash Equilibrium, we can conclude that it is a “**No Regret**” solution for any game, but not necessarily the most optimal one.

Introduction to Game Trees



A Game Tree is a graphical representation of every possible sequence of moves in a game.

Each branch represents a possible action.

Each node represents a game state.

Leaf nodes represent the final outcome with an associated utility value.

Components

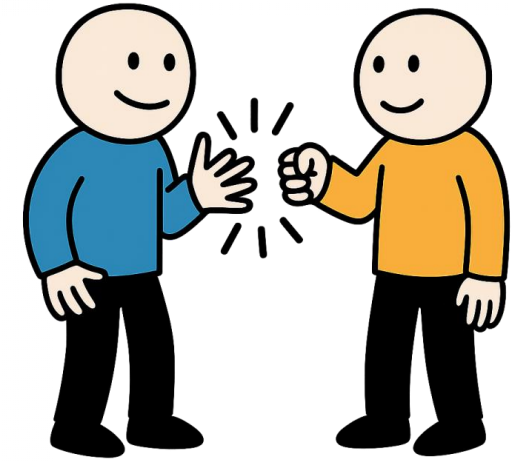
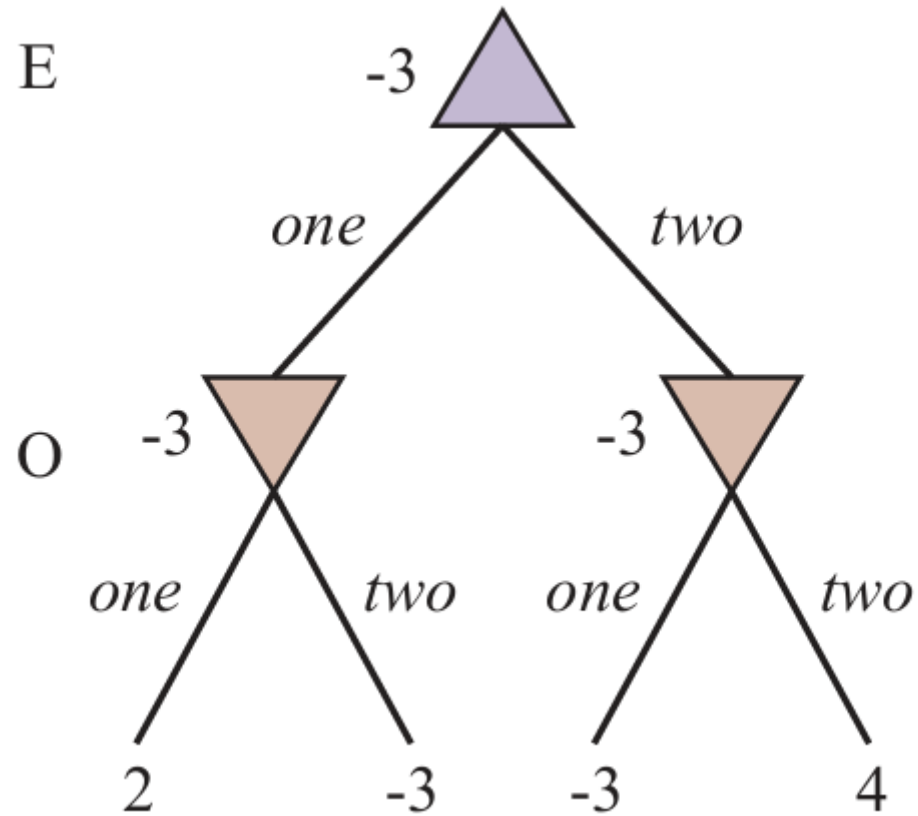
- Root Node
- Decision Nodes
- Branches
- Terminal Nodes
- Utility Values

Game trees allow AI agents to evaluate future consequences before selecting an action.

Advantages

- Visualizes decision making.
- Supports optimal planning.
- Forms the basis of Minimax search.

Introduction to Game Trees



Understanding the Minimax Principle



The Minimax algorithm is used in competitive games involving two players.

Basic Idea

One player tries to maximize the utility.

The opponent tries to minimize that utility.

Both players are assumed to play optimally.

Minimax Procedure

Step 1: Generate the complete game tree.

Step 2: Assign utility values to terminal nodes.

Step 3: MIN chooses the minimum utility among its children.

Step 4: MAX chooses the maximum value returned by MIN.

The selected move guarantees the **best worst-case outcome**.

Understanding the Minimax Principle

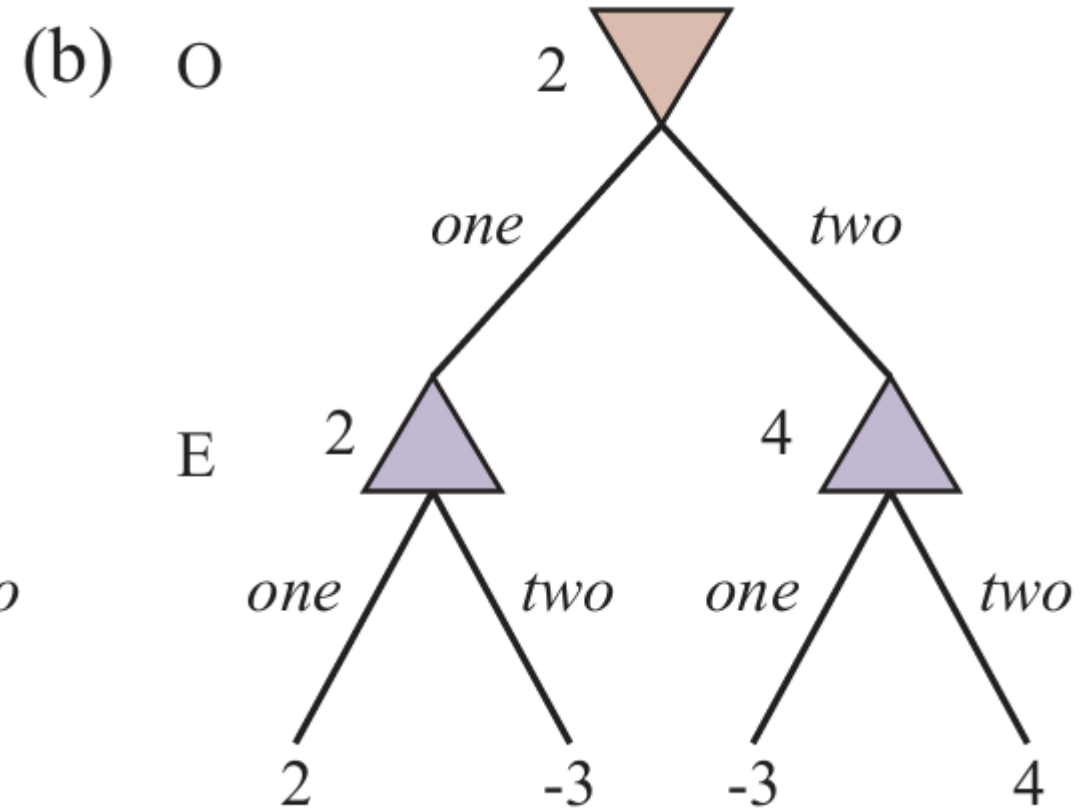
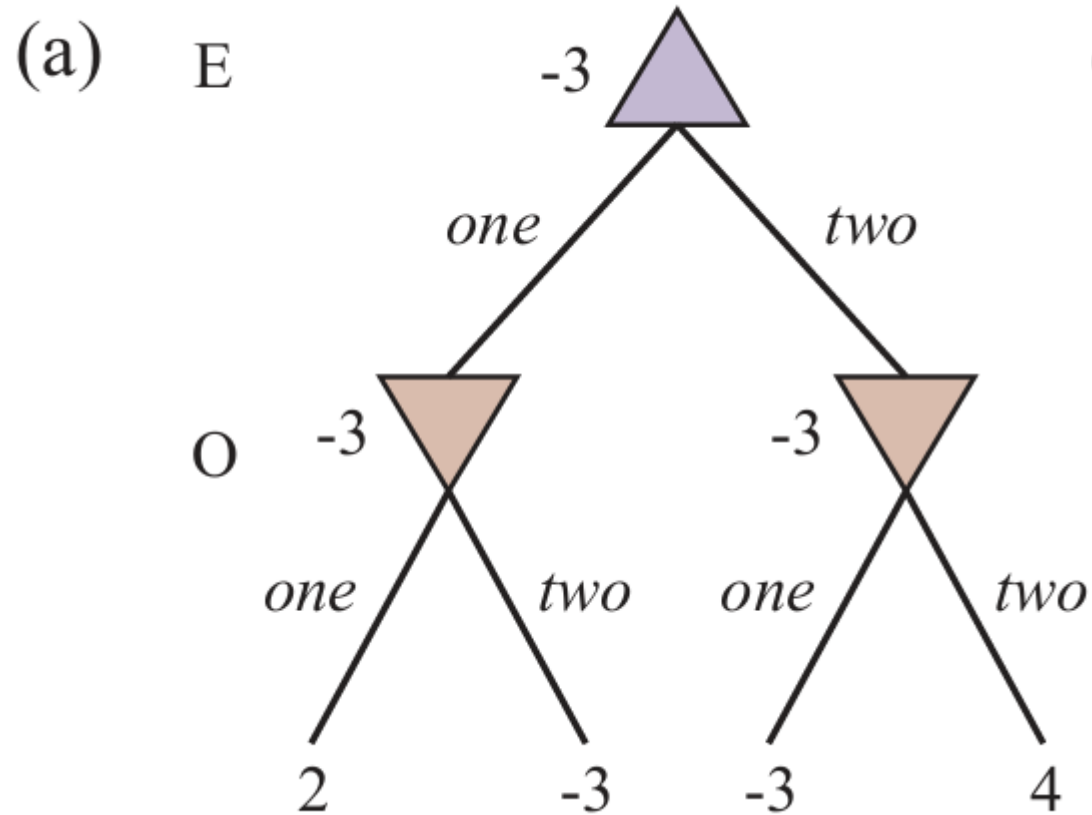


Figure (a) and (b): Minimax game trees for two-finger Morra if the players take turns playing pure strategies.

Minimax guarantees the optimal decision when both players are perfectly rational and play optimally.

Minimax Evaluation: Step-by-Step Example



The Minimax algorithm determines the best move by assuming that both players make optimal decisions. The maximizing player (MAX) attempts to obtain the highest possible payoff, while the minimizing player (MIN) attempts to reduce MAX's payoff.

Steps in Minimax Evaluation

Step 1: Construct the complete game tree.

Step 2: Assign utility values to all terminal (leaf) nodes.

Step 3: Evaluate all MIN nodes by selecting the smallest utility value among their children.

Step 4: Propagate these values upward.

Step 5: Evaluate MAX nodes by selecting the largest value received from the MIN nodes.

Step 6: The value at the root represents the optimal outcome assuming both players act rationally.

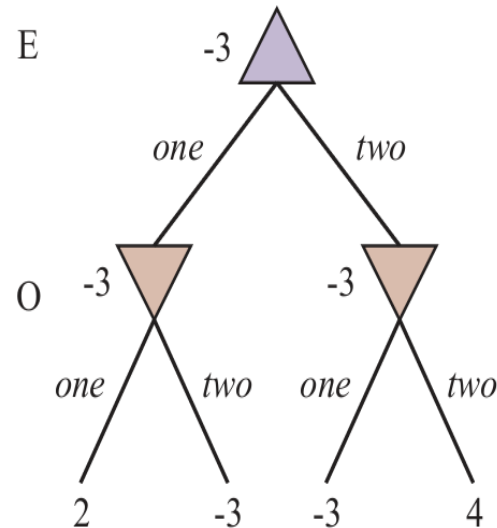
Minimax Evaluation: Step-by-Step Example



Why is Minimax Important?

- Guarantees the best possible decision in adversarial environments.
- Prevents worst-case outcomes.
- Widely used in AI game-playing systems.

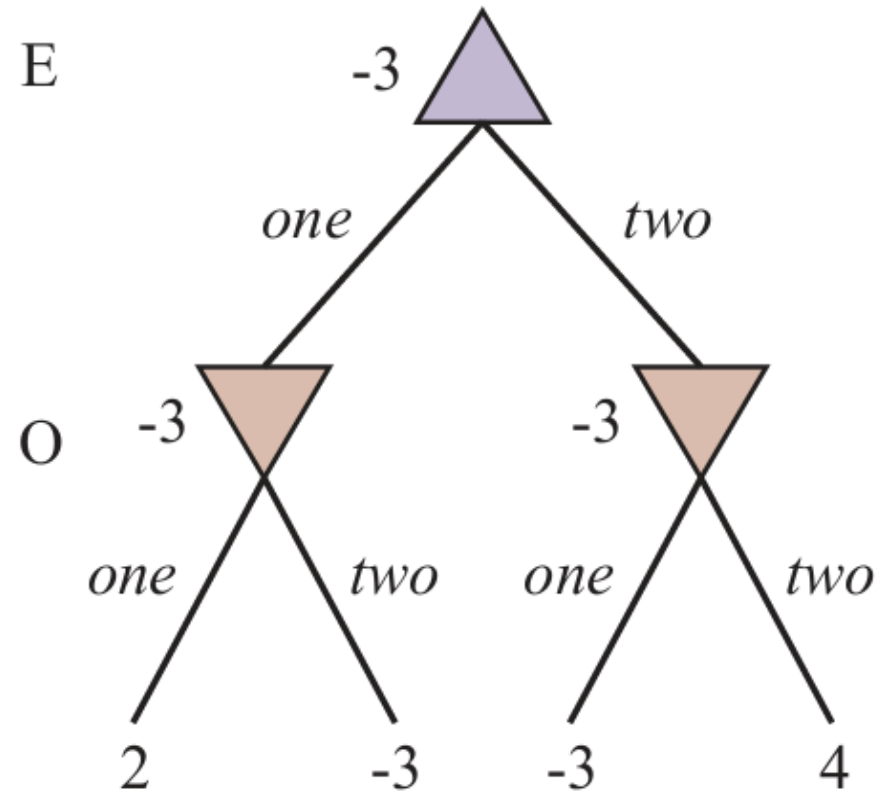
The algorithm never assumes that the opponent will make mistakes.



Understanding the Minimax Decision



Consider the game tree shown in Figure



Understanding the Minimax Decision



- The terminal nodes represent the utilities obtained after all moves have been completed.
- The MIN player evaluates each subtree and selects the smallest utility because its objective is to reduce MAX's reward.
- After MIN completes its evaluation, MAX compares the returned values and selects the branch with the highest utility.

Example

Suppose the left subtree returns -3 and the right subtree also returns -3 .

MAX has no opportunity to improve the outcome and therefore receives a utility of -3 .

This represents the guaranteed payoff regardless of the opponent's actions.

Interpretation

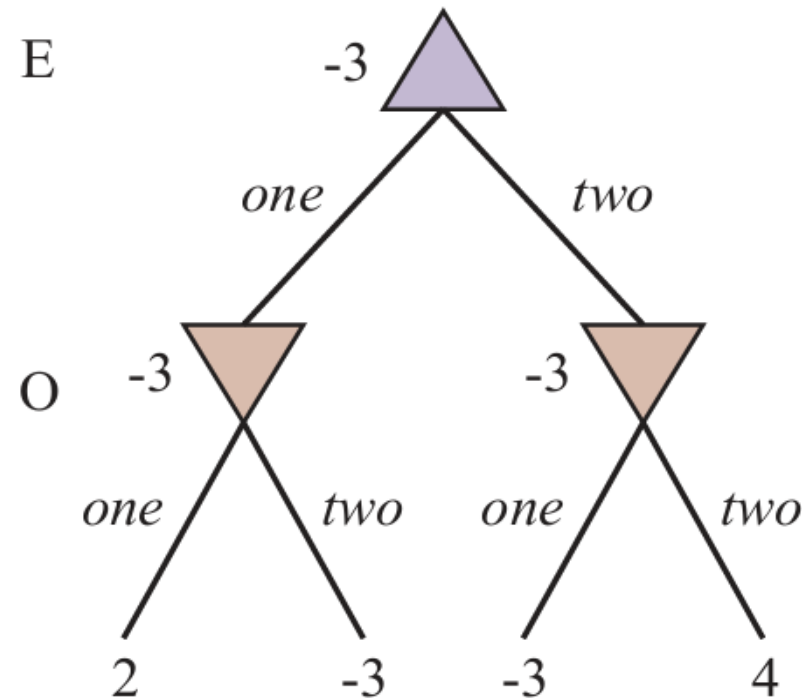
Even if the final utility is negative, Minimax still selects the best available option among all possible worst-case outcomes.

Understanding the Minimax Decision



Why doesn't MAX simply choose the branch with the highest leaf value?

Answer: Because MIN controls the next move.



Limitations of Pure Strategies



- Until now, we assumed that every player always chooses one specific action.
- Such strategies are called **Pure Strategies**.
- However, pure strategies are not always optimal.

Problem

- If an opponent can predict your next move, they can exploit your strategy.

Example

- Imagine a goalkeeper who always dives to the left during a penalty kick.
- After observing this pattern, the striker will always shoot to the right.
- The predictable strategy becomes ineffective.

Therefore,

- AI agents often require strategies that introduce uncertainty.
- This leads to the concept of **Mixed Strategies**.

Instead of choosing one action every time, an intelligent agent chooses among multiple actions with certain probabilities.

Mixed Strategy



- A **Mixed Strategy** is a probability distribution over multiple pure strategies.
- Instead of always selecting the same action, the player randomly selects an action according to predefined probabilities.

Example

Suppose a player has two actions:

- Action One
- Action Two

Rather than always choosing Action One, the player may choose:

- Action One → 70% probability
- Action Two → 30% probability

This randomness prevents the opponent from accurately predicting future actions.

Mixed Strategy



Advantages

Reduces predictability.

Increases robustness against intelligent opponents.

Often produces better long-term performance.

Applications

Poker

Security patrol scheduling

Cybersecurity

Military planning

Autonomous robotics

Why Mixed Strategies Work



The purpose of a mixed strategy is **not randomness for its own sake**.

Instead, randomness is used strategically to prevent exploitation.

Real-Life Example

Consider a football penalty kick.

If the goalkeeper always dives left, the striker will easily score.

Instead, the goalkeeper randomizes:

- Left $\rightarrow$ 40%
- Right $\rightarrow$ 40%
- Center $\rightarrow$ 20%

Now the striker cannot confidently predict the goalkeeper's action.

AI Perspective

Many intelligent systems intentionally introduce randomness to make their behavior less predictable.

Important Concept

A mixed strategy makes every possible action appear equally risky to the opponent.

Mixed Strategy in Morra Game



The Morra game demonstrates situations where pure strategies fail.

Suppose Player E chooses

- **One** with probability p
- **Two** with probability $(1 - p)$

The opponent now faces uncertainty.

The expected payoff depends on the probability value chosen.

Instead of evaluating a single move, the game evaluates the expected utility of every possible probability.

Objective

Find the probability that maximizes the minimum expected payoff.

This is called the **Maximin Mixed Strategy**.

Observation

The player's decision is no longer a single action.

The decision itself becomes choosing the **best probability**.

Mixed Strategy in Morra Game

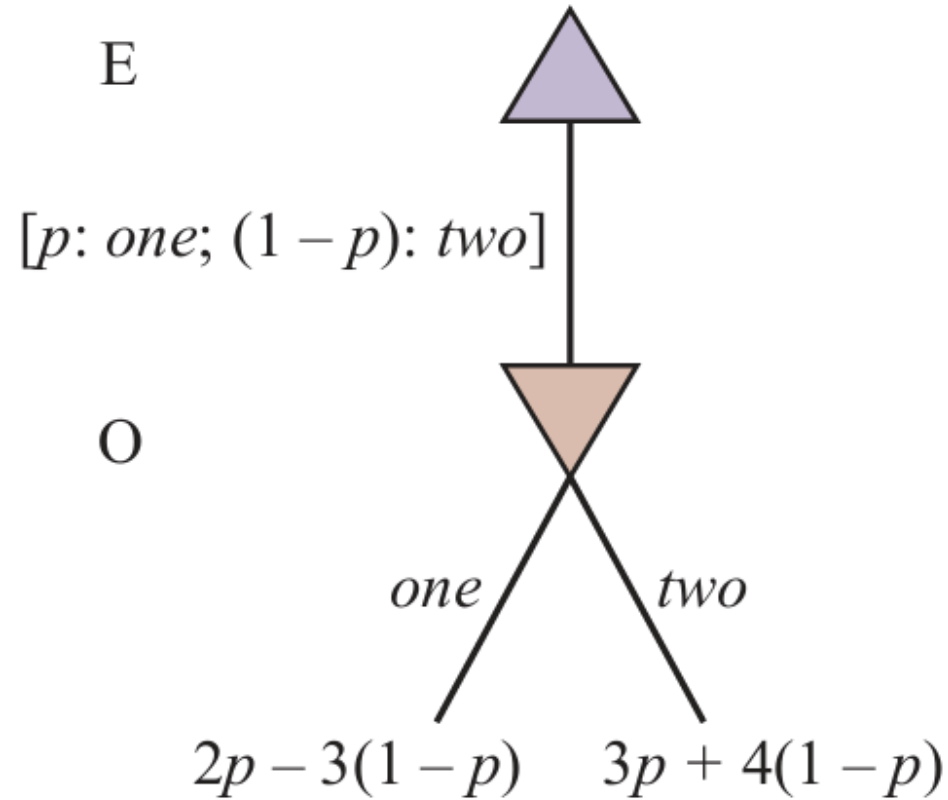


Figure Parameterized game trees where the first player plays a mixed strategy.

Parameterized Game Trees



Parameterized game trees extend the Minimax algorithm by replacing fixed decisions with probabilities. Instead of selecting one branch, the player distributes choices among several branches. The utility now depends on a probability parameter.

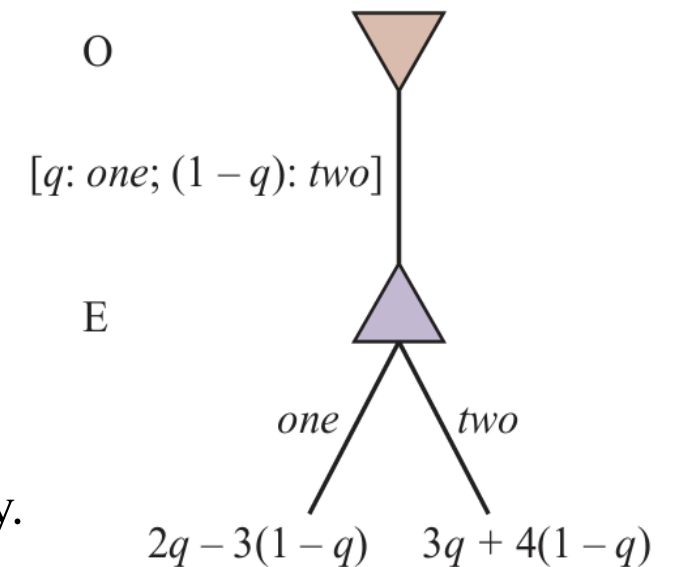
Example

Suppose

- Choose "One" with probability q
- Choose "Two" with probability $1 - q$

The resulting payoff becomes a mathematical function of q .

The objective is to determine the value of q that maximizes the guaranteed utility.



Importance

Parameterized game trees allow AI agents to optimize uncertain decisions mathematically.

Maximin Strategy



The **Maximin Strategy** selects the action that produces the highest minimum payoff.

Mathematical Interpretation

For every possible strategy,

1. Determine the minimum payoff.
2. Compare all minimum values.
3. Choose the largest among them.

This guarantees the best possible outcome even under the worst circumstances.

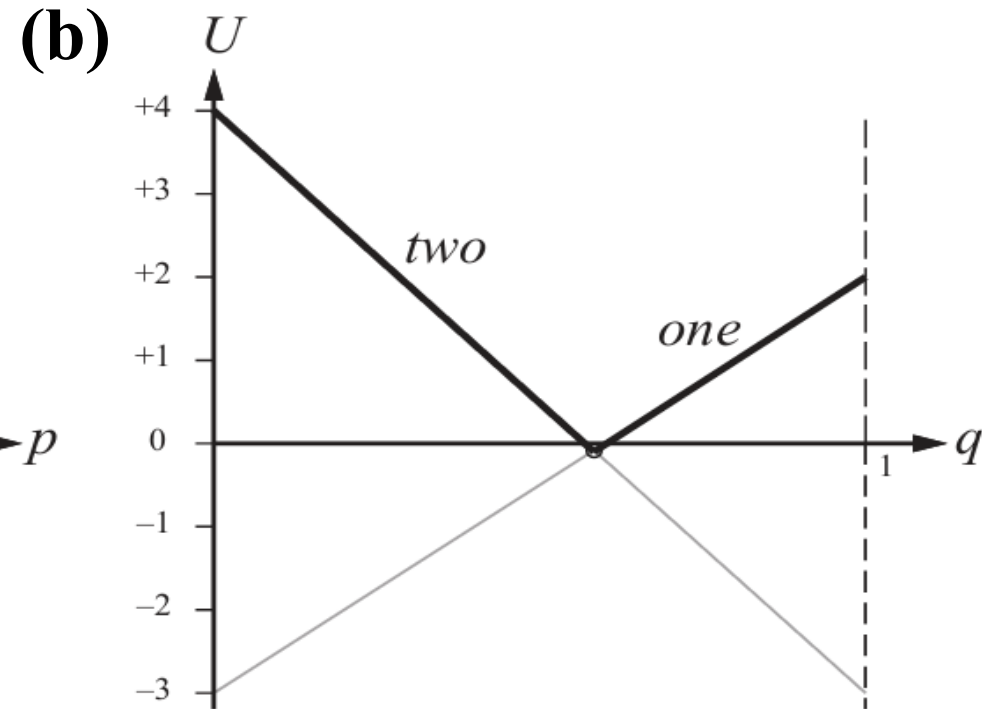
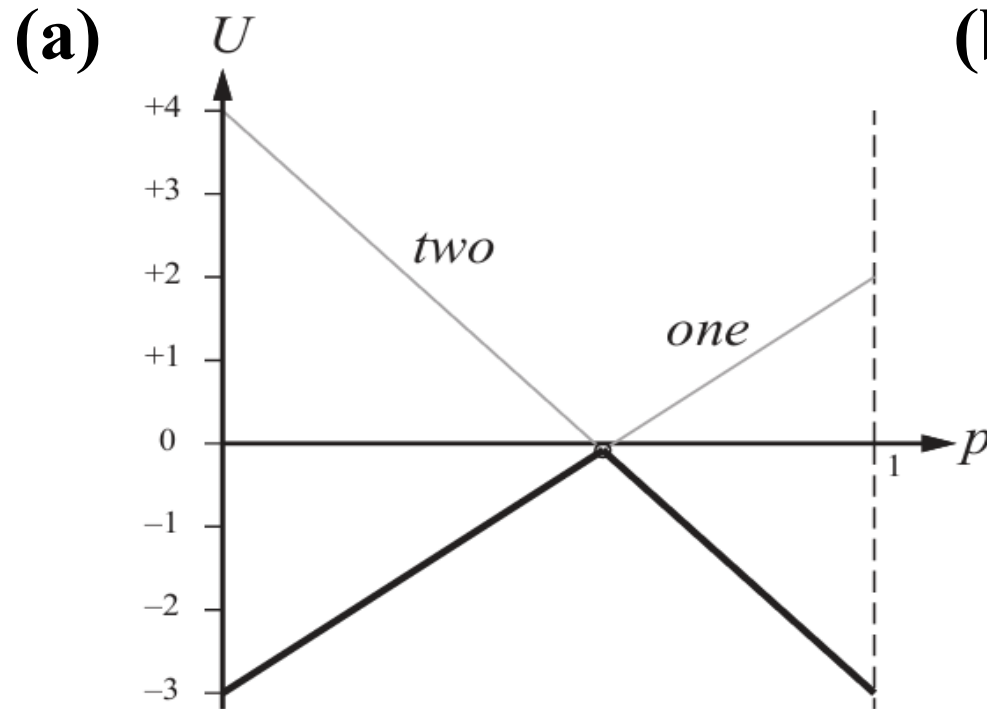
Why Maximin? - It protects the player against highly intelligent opponents.

AI Applications

- Chess engines
- Military planning
- Financial risk management
- Autonomous systems

Maximin is a conservative decision-making strategy that minimizes risk.

Graphical Interpretation of Maximin



For any particular value of the probability parameter, the second player will choose the “better” of the two actions, so the value of the first player’s mixed strategy is given by the heavy lines. The first player will choose the probability parameter for the mixed strategy at the intersection point.

Graphical Interpretation of Maximin



The graph illustrates how the expected payoff changes with different probability values. Each line corresponds to one of the opponent's possible strategies. The player seeks the probability where the lowest payoff is as high as possible.

Important Observation

The optimal probability occurs where the two payoff lines intersect. At this point:
Neither strategy dominates.
The opponent cannot exploit predictable behavior.
The minimum payoff is maximized.

Interpretation

The intersection point represents the optimal mixed strategy.

Repeated Games



Many real-world interactions are **not played only once**.

Instead, the same players interact repeatedly.

Examples include:

- Business competition
- Online marketplaces
- Cybersecurity
- Human relationships
- Autonomous vehicle coordination

Characteristics of Repeated Games

- Previous actions influence future decisions.
- Players learn from experience.
- Reputation becomes important.
- Cooperation may emerge over time.

Difference Between One-Shot and Repeated Games



One-Shot Game	Repeated Game
Played once	Played multiple times
No history	History influences future actions
Short-term strategy	Long-term strategy
Limited cooperation	Cooperation often develops

Repeated games require strategies that remember previous interactions. One powerful way to model such behavior is through **Finite State Machines**.

Repeated Games



- Unlike one-shot games, repeated games involve the same players interacting multiple times. Since players remember previous actions, their current decisions are influenced by past experiences and future expectations.
- In repeated games, players are not only concerned with maximizing immediate rewards but also with maintaining long-term benefits. As a result, cooperation may emerge even when short-term incentives encourage competition.

Characteristics of Repeated Games

- Players interact multiple times.
- Previous actions influence future decisions.
- Players can reward cooperation.
- Players can punish betrayal.
- Strategies evolve over time.

Examples

- Business price competition
- Autonomous vehicle coordination
- Network routing protocols
- Robot collaboration
- Online marketplaces

AI Perspective: Repeated interaction allows intelligent agents to **learn**, **adapt**, and **improve** their strategies over time.

Why Memory Matters in Repeated Games



- In repeated games, every action creates a history that influences future interactions.
- Suppose two autonomous delivery robots repeatedly share a narrow corridor.
- If both cooperate, traffic flows smoothly.
- If one behaves selfishly, the other may refuse to cooperate in future encounters.
- Thus, intelligent agents begin to develop trust and reputation.

Benefits of Memory

- Encourages cooperation.
- Discourages selfish behavior.
- Builds trust among agents.
- Enables long-term optimization.

AI Applications

- Multi-agent reinforcement learning
- Cloud resource allocation
- Swarm robotics
- Autonomous transportation

Memory transforms isolated decisions into long-term strategic planning.

Strategies in Repeated Games



- A repeated game strategy determines how a player reacts based on previous outcomes.
- Instead of making isolated decisions, the player continuously updates its behavior according to the opponent's actions.

Some commonly used strategies include:

- Hawk Strategy
- Dove Strategy
- Grim Strategy
- Tit-for-Tat Strategy
- Tat-for-Tit Strategy

Each strategy represents a different balance between cooperation and competition.

Think About It

If an opponent always cooperates, should an intelligent agent always cooperate as well?

This question motivates the study of strategic behavior in repeated interactions.

Hawk and Dove Strategies



The Hawk–Dove model represents conflict between aggressive and cooperative behaviors.

Hawk Strategy

A Hawk player is always aggressive.

Characteristics:

- Never compromises.
- Always competes.
- Attempts to maximize immediate payoff.
- Can cause conflicts when facing another Hawk.

Dove Strategy

A Dove player prefers cooperation.

Characteristics:

- Avoids unnecessary conflict.
- Shares resources.
- Seeks mutual benefit.
- May be exploited by aggressive opponents.

Real-World Example

Consider two autonomous vehicles approaching a narrow bridge.

- Hawk behavior: Neither vehicle gives way.
- Dove behavior: One vehicle yields, allowing smooth traffic flow.

Most intelligent systems require a balance between aggression and cooperation rather than following one extreme.

Grim Strategy



The Grim Strategy is one of the strictest strategies used in repeated games.

Rule

1. Cooperate initially.
2. Continue cooperating as long as the opponent cooperates.
3. If the opponent defects even once, defect forever.

Advantages

- Strongly discourages betrayal.
- Encourages long-term cooperation.
- Simple to implement.

Disadvantages

- Extremely unforgiving.
- A single accidental mistake permanently destroys cooperation.
- Not suitable for noisy environments where communication errors occur.

Example

Two companies agree to maintain fair pricing.

If one company violates the agreement once, the other permanently enters aggressive competition.

AI Perspective: Grim Strategy is effective when agents can perfectly observe each other's actions.

Tit-for-Tat Strategy



Tit-for-Tat is one of the most successful strategies discovered in Game Theory.

Rule

- Start by cooperating.
- Afterwards, simply repeat whatever the opponent did in the previous round.

Behavior

Opponent cooperates → Cooperate

Opponent defects → Defect once

Opponent cooperates again → Return to cooperation

Why is Tit-for-Tat Successful?

It is:

Nice (never defects first)

Forgiving (returns to cooperation)

Retaliatory (punishes betrayal)

Easy to understand

Tit-for-Tat Strategy



AI Applications

Network communication protocols

Robot collaboration

Distributed computing

Multi-agent reinforcement learning

Why is Tit-for-Tat often more successful than Grim Strategy?

Answer: It forgives occasional mistakes.

Tat-for-Tit Strategy



Tat-for-Tit is the opposite behavioral approach.

Instead of immediately retaliating, the player initially defects and then imitates the opponent's previous move.

Characteristics

Less cooperative initially.

Can delay trust formation.

May increase conflicts.

Suitable for environments where deception is common.

Comparison

Tit-for-Tat	Tat-for-Tit
Begins with cooperation	Begins with defection
Builds trust quickly	Builds trust slowly
Encourages cooperation	More defensive
Widely used	Less common

Modern AI systems generally prefer Tit-for-Tat because it promotes stable cooperation.

Finite State Machine Strategies



Repeated game strategies can be represented using **Finite State Machines (FSMs)**.

A Finite State Machine consists of:

- States
- Transitions
- Inputs
- Outputs

Each state represents the current strategy of the player.

The next state depends on the opponent's previous action.

Why FSM?

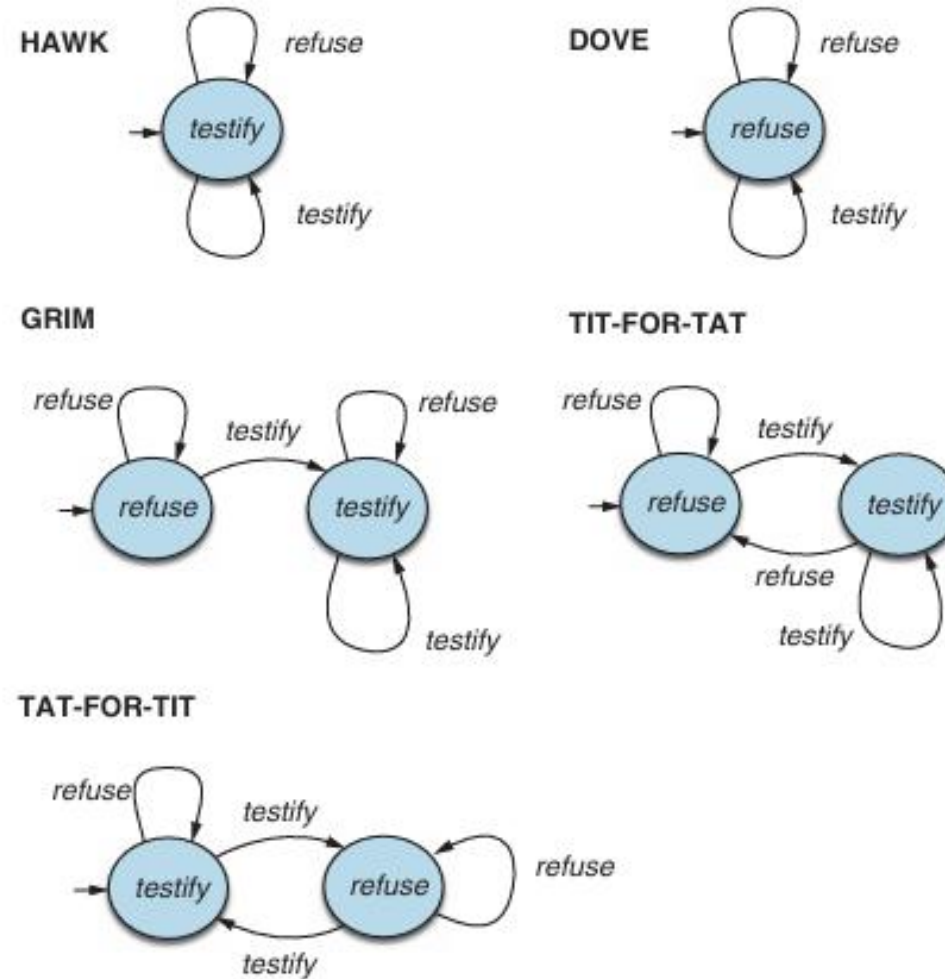
Finite State Machines provide a compact way to model memory-based strategies.

Instead of remembering the entire history, the agent only remembers its current state.

Applications

- Communication protocols
- Robotics
- Autonomous agents
- Industrial automation

Finite State Machine Strategies



Some common, colorfully named finite-state machine strategies for the in finitely repeated prisoner's dilemma 56

Example of FSM-Based Strategies



Consider a Tit-for-Tat player represented as a Finite State Machine.

- Initially, State $\rightarrow$ Cooperate
- If the opponent cooperates, Remain in the Cooperate state.
- If the opponent defects, Transition to the Defect state.
- Once the opponent resumes cooperation, Return to the Cooperate state.

Advantages of FSM Representation

Easy implementation.

Minimal memory.

Fast decision making.

Suitable for embedded AI systems.

AI Example

Many robot soccer teams implement behavioral controllers using Finite State Machines because they are efficient and easy to interpret.

Comparison of Repeated Game Strategies



Strategy	First Move	Reaction to Defection	Forgiveness	Suitable Environment
Hawk	Defect	Always Defect	No	Highly Competitive
Dove	Cooperate	Continue Cooperating	Yes	Cooperative Systems
Grim	Cooperate	Defect Forever	No	Stable Environments
Tit-for-Tat	Cooperate	Defect Once	Yes	Most Real-World Systems
Tat-for-Tit	Defect	Mimics Opponent	Limited	Defensive Scenarios

- No strategy is universally optimal.
- The best strategy depends on the environment.
- Cooperation often emerges through repeated interaction.
- Intelligent agents balance competition and collaboration.
- FSMs provide an efficient way to implement repeated-game strategies.

Sequential Games



Unlike simultaneous games, **sequential games** involve players making decisions one after another. Each player observes the previous actions (fully or partially) before making the next move. This allows future decisions to depend on earlier actions.

Sequential games are widely used to model situations where timing and order of decisions are important.

Characteristics of Sequential Games

- Players move one after another.
- Later players can observe earlier decisions (in perfect information games).
- The order of play influences the outcome.
- Every decision changes the future state of the game.

Examples

- Chess
- Negotiation between companies
- Investment decisions
- Political campaigns
- Supply chain management

Extensive Form Representation



Sequential games are represented using the **Extensive Form**, which provides a graphical representation of the entire decision process.

Components of an Extensive Form Game

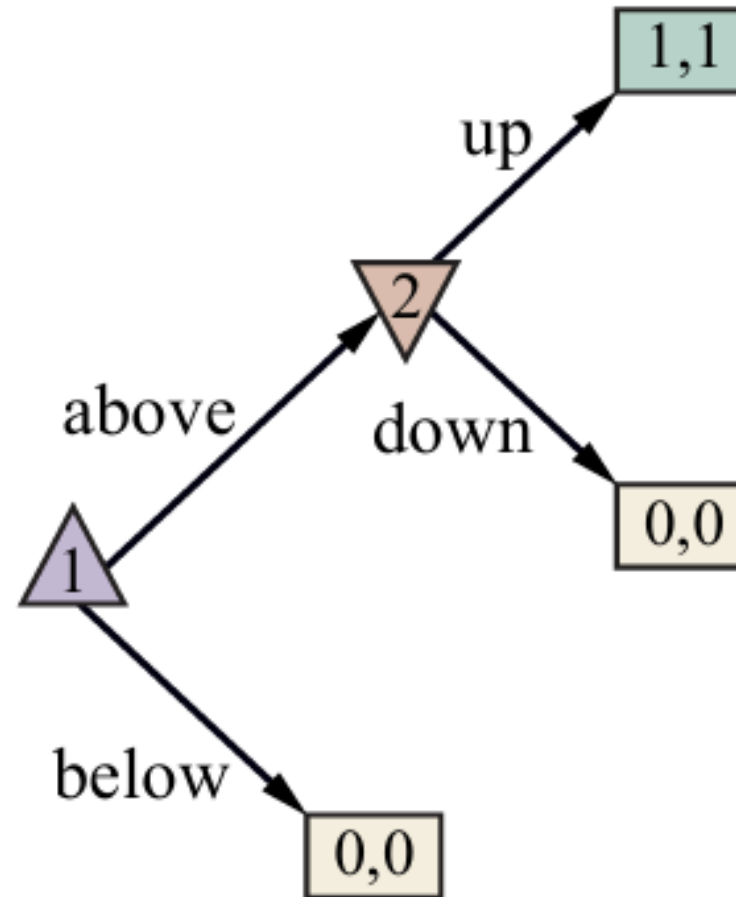
- **Decision Nodes:** Points where a player chooses an action.
- **Branches:** Possible actions available.
- **Terminal Nodes:** Final outcomes of the game.
- **Payoff Values:** Rewards received by each player.
- **Information Sets:** Represent what players know when making decisions.

Unlike a payoff matrix, an extensive-form game clearly illustrates the **sequence of decisions**.

Advantages

- Models complex strategic interactions.
- Represents dynamic decision-making.
- Captures uncertainty and incomplete information.
- Suitable for AI planning algorithms.

Extensive Form Representation



An extensive-form game with a counterintuitive Nash equilibrium

Backward Induction



Backward Induction is one of the most powerful methods for solving sequential games.

Basic Idea

Instead of analyzing the game from the beginning, start from the **last decision** and work backward toward the initial move.

Steps

Identify the terminal nodes.

Determine the optimal decision at the last stage.

Replace each subtree with its optimal payoff.

Continue moving backward until reaching the root.

Why Does It Work?

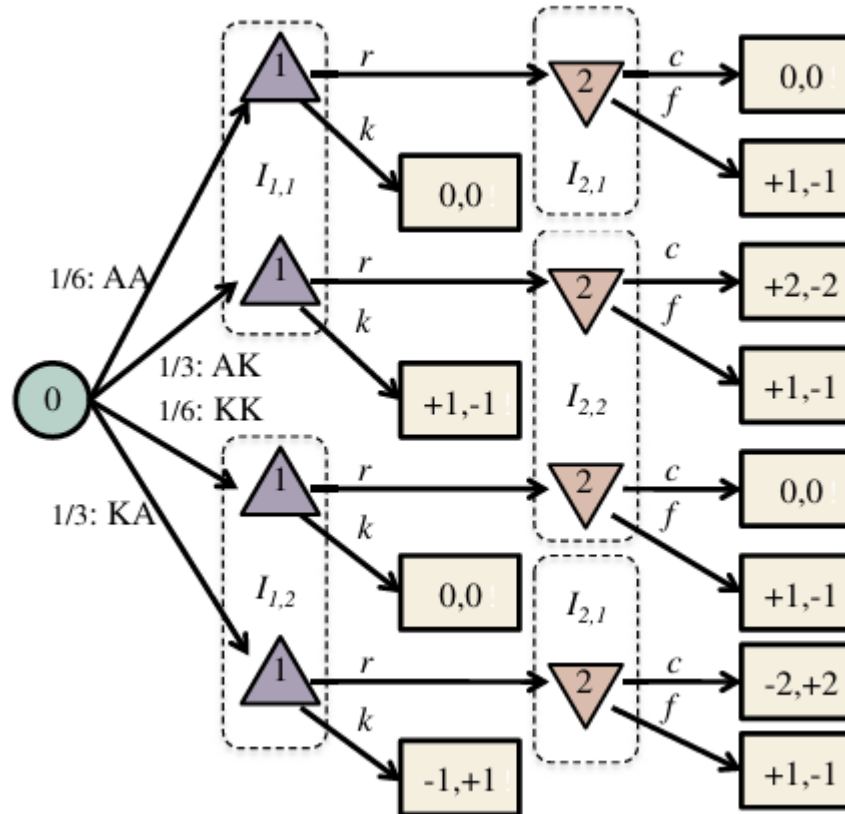
Every rational player chooses the action that maximizes their payoff at every stage.

Therefore, earlier players can predict future decisions and plan accordingly.

Backward Induction



Extensive form of a simplified version of poker with two players and only four cards. The moves are r (raise), f (fold), c (call), and k (check)



AI Applications

- Route planning
- Robotics
- Strategic planning
- Multi-agent systems
- Business decision support

Subgame Perfect Nash Equilibrium (SPNE)



- A **Subgame Perfect Nash Equilibrium (SPNE)** is a refinement of Nash Equilibrium designed specifically for sequential games.
- A strategy profile is a Subgame Perfect Nash Equilibrium if it forms a Nash Equilibrium in **every subgame** of the original game.

Why Do We Need SPNE?

Some Nash Equilibria contain **non-credible threats**—players may threaten actions they would never actually take because they are not in their best interest.

SPNE eliminates such unrealistic strategies by ensuring that every decision remains rational at every stage of the game.

Properties

- Always based on rational behavior.
- Removes non-credible threats.
- Obtained using Backward Induction.
- Produces stable solutions.

SPNE enables intelligent agents to make decisions that remain optimal throughout the entire interaction.

Games with Imperfect Information



Not all games provide complete knowledge of previous actions.

In **Imperfect Information Games**, players must make decisions without knowing the complete state of the game.

Characteristics

- Hidden information.
- Uncertainty about opponents' actions.
- Decisions based on probabilities and beliefs.
- Multiple possible future outcomes.

Examples

- Poker
- Card games
- Auctions
- Cybersecurity
- Military operations

Games with Imperfect Information



AI Challenges

An AI agent must estimate unknown information using:

- Probability theory
- Bayesian reasoning
- Opponent modeling
- Learning from previous observations

Perfect Information	Imperfect Information
All moves visible	Some information hidden
Deterministic planning	Probabilistic reasoning
Chess	Poker

Poker: A Game of Imperfect Information



Poker is one of the best-known examples of an imperfect information game.

Unlike chess, players cannot see their opponents' cards.

Why Poker is Challenging

Success depends on:

- Hidden information
- Probability calculations
- Bluffing
- Opponent modeling
- Risk assessment

AI Techniques Used

- Bayesian Networks
- Monte Carlo Tree Search (MCTS)
- Reinforcement Learning
- Deep Neural Networks
- Counterfactual Regret Minimization (CFR)

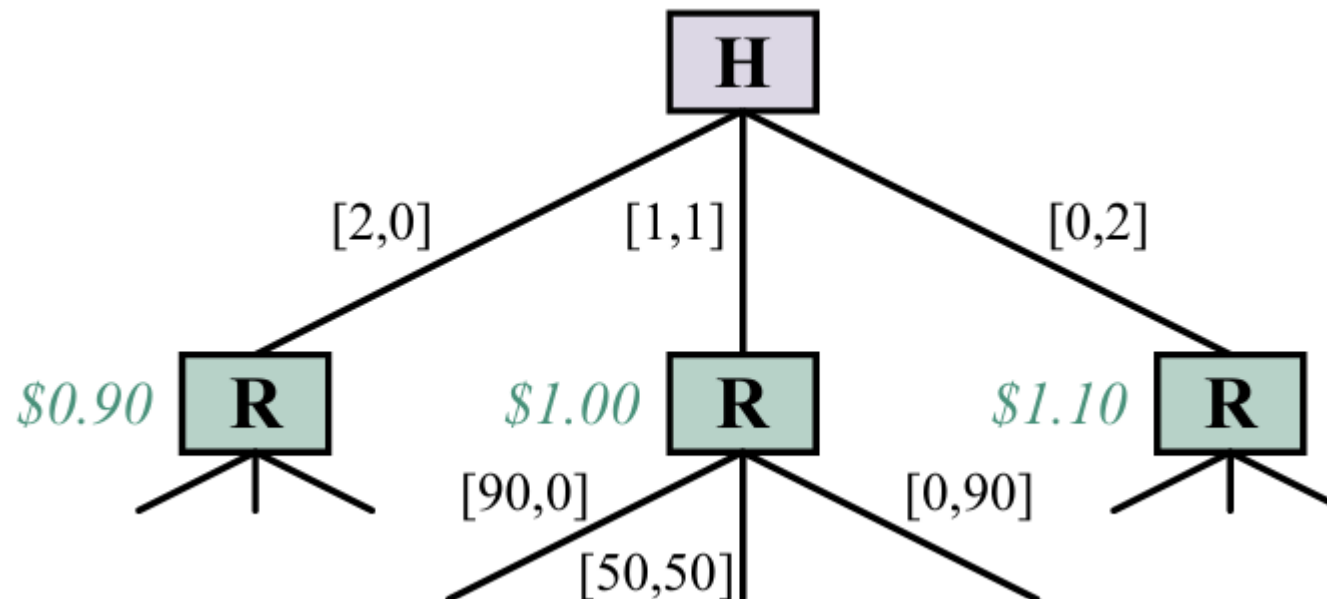
Backward Induction



Modern AI systems such as Pluribus and Libratus have demonstrated superhuman performance in multiplayer poker, showcasing AI's ability to handle uncertainty and strategic reasoning.

Why is Poker considered more difficult for AI than Chess?

Answer: Because players must make decisions with incomplete information.



Assistance Games (Paperclip Game)



An **Assistance Game** is a cooperative game where an AI agent assists a human in achieving a shared objective.

Unlike competitive games, the agent and the human have **aligned goals**, but the AI may not know the human's exact preferences.

Characteristics

Human and AI collaborate.

Shared objective.

AI learns human preferences over time.

Communication and observation are important.

Paperclip Game Example

In the Paperclip Game, a human and a robot work together to collect paperclips.

Initially, the robot is uncertain about the human's preferences.

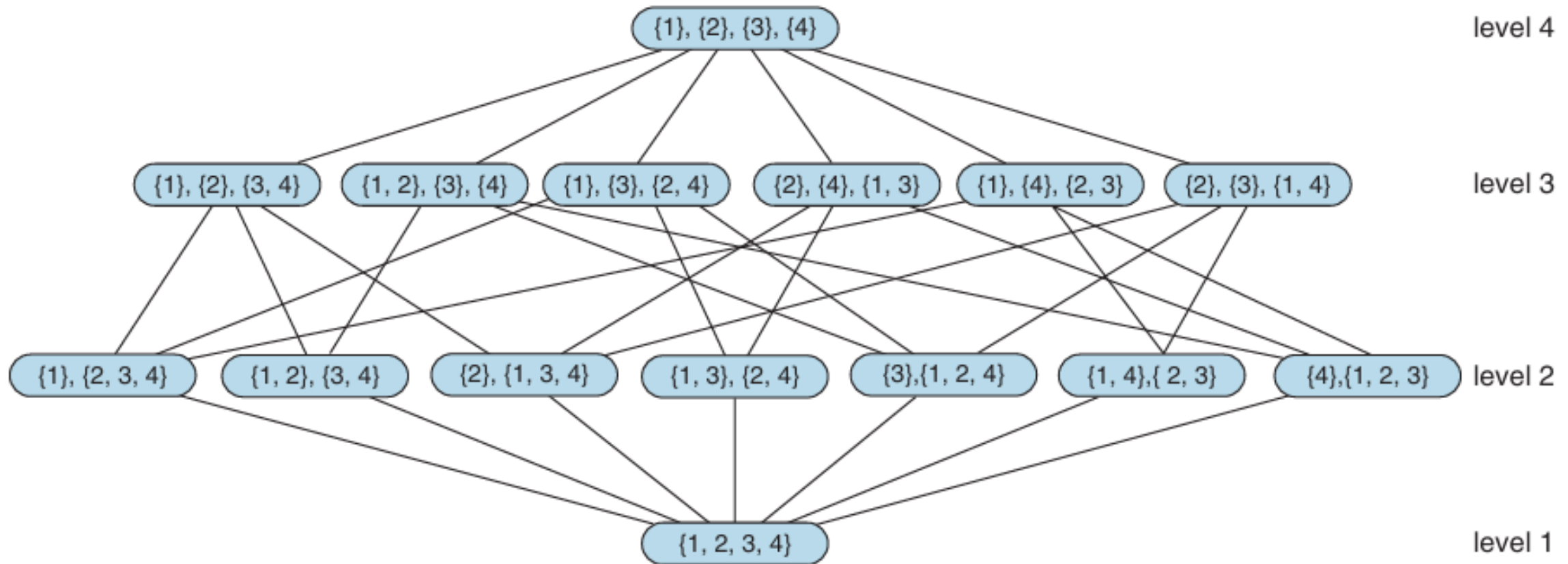
By observing actions and receiving feedback, the robot gradually learns:

Which tasks are preferred.

How to assist effectively.

When to seek clarification.

Backward Induction



The coalition structure graph for $N = \{1, 2, 3, 4\}$. Level 1 has coalition structures containing a single coalition; level 2 has coalition structures containing two coalitions, and so on

Backward Induction



Applications

- Healthcare assistants
- Autonomous robots
- Personal digital assistants
- Smart homes
- Human–robot collaboration

Modern AI systems increasingly focus on cooperation, where success depends on understanding and assisting human users rather than competing against them.